ML Challenge 2025: Smart Product Pricing

Solution Report

Team Name: DataVengers
Team Members: Thammisetty Srilekha, Udit Jain, Ujjawal Sharma
Submission Date: October 12, 2025

1. Executive Summary

We implement a **rule-compliant** text+structured approach for product price prediction from `catalog_content`. Text is vectorized with a hybrid **HashingVectorizer + TF-IDF** (1–3 grams), and we engineer structured features (quantity/unit normalization, text statistics, optional image_link tokens). Numerical features are scaled; categorical tokens are one-hot encoded. We train two regressors—**Ridge** (on log-price) and **XGBoost**—and **blend** them using inverse-RMSE weights. Features are cached; the final model is persisted to reuse for later inference.

2. Methodology Overview

2.1 Problem Analysis

Supervised regression from catalog descriptions where pack sizes, units (ml/g/count), and textual patterns correlate strongly with price.

2.2 Approach

- **Structured features:** Regex-driven extraction of *pack_of*, *value_num* + unit normalization to base units (g/ml/count), derived logs (value_base, value_per_pack), and text stats (*len, words, digits, upper, punct* and ratios). From `image_link` we derive simple tokens (*ext, host, has_dim, len*).
- **Vectorization:** Hashing (1–2g) + TF-IDF (1–3g, up to 120k feats) on truncated text ($\leq$ MAX_CHARS). Numeric features: `StandardScaler(with_mean=False)`. Categoricals: `OneHotEncoder(handle_unknown=ignor min_frequency=20)`.
- **Modeling:** Train **Ridge** and **XGBRegressor** on log(1 + price); exponentiate for predictions. Blend via inverse-RMSE weights on a holdout split.
- **Validation/holdout:** 90/10 `ShuffleSplit` holdout for final reporting; optional 5-fold CV logging.
- **Caching & reuse:** Sparse feature matrices cached to disk; trained models + blend weights saved to `final_model.pkl`. If present, we skip training and directly infer.

3. Model Architecture

3.1 Diagram

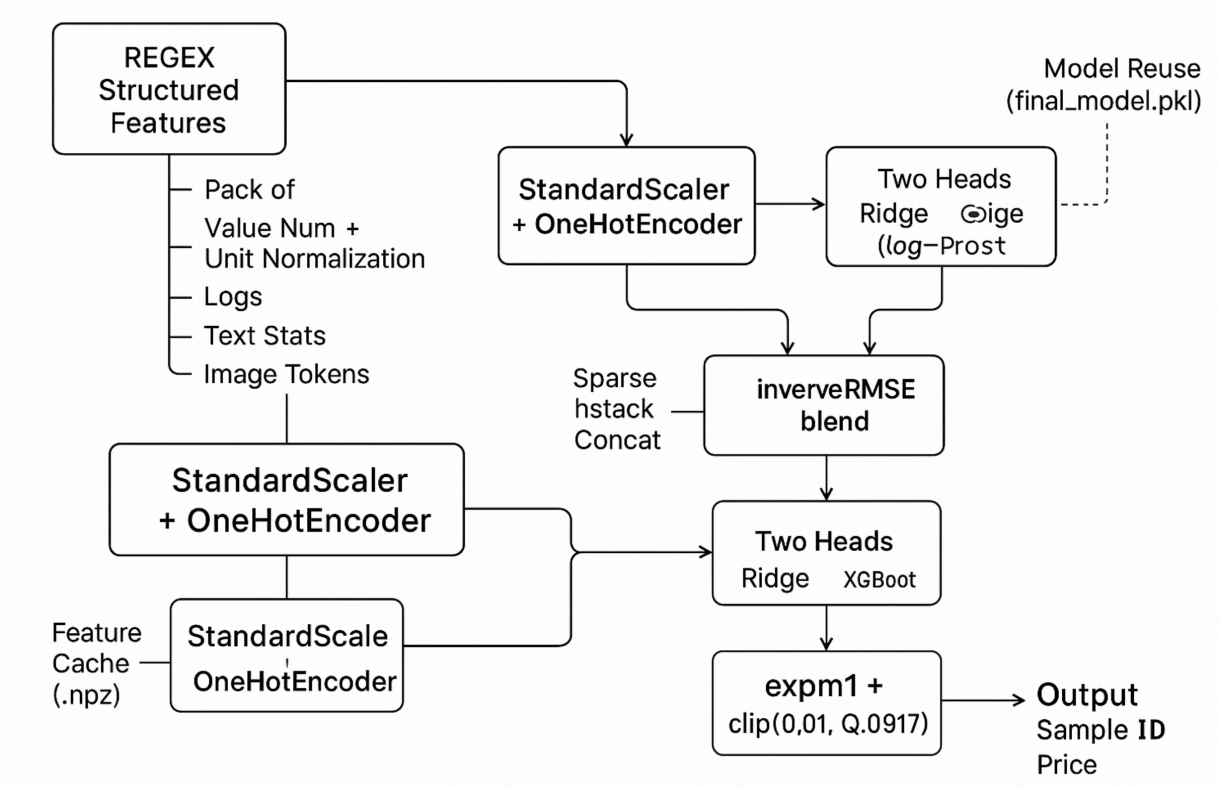

Figure 1: Pipeline — regex struct feats + unit normalization, Hashing + TF-IDF, scaling & OHE, sparse concat, Ridge & XGBoost heads, inverse-RMSE blending, price output.

3.2 Key Implementation Details

- **Text limits:** `MAX_CHARS` (env) trims long descriptions for memory safety.
- **XGBoost (key params):** `n_estimators=1500`, `learning_rate=0.05`, `max_depth=9`, `subsample=0.85`, `colsample_bytree=0.75`, `tree_method=hist`, `eval_metric=rmse`, seed-aligned.
- **Blending:** weights $w_i \propto 1/\text{RMSE}_i^2$ (normalized) computed on holdout.
- **Post-processing:** gentle clipping of predictions to $[0.01, Q_{0.997}]$.
- **Reproducibility:** seeds fixed; logs include per-model metrics and blend weights.

4. Dataset, I/O and Compliance

4.1 Dataset Columns

- **sample_id** (unique row id), **catalog_content** (string), **image_link** (optional), **price** (train only).

4.2 Output Format

CSV with exactly: `sample_id`, `price`. One prediction per test row. The submission file written by the script is `test_out.csv`.

4.3 Rule Compliance

- No external price lookup / web augmentation.
- Only `catalog_content`-derived features (plus benign `image_link` tokens) are used.
- No third-party calls during training/inference; all resources are local.

5. Training Objective and Metric

Training objective: models fit $\mathrm{MSE}\big(\log(1+\hat{y}), \log(1+y)\big)$.

Reporting metrics: SMAPE, RMSE, MAE, and R^2 are computed on the holdout. SMAPE is:

$$\mathrm{SMAPE} = 100 \times \frac{1}{N} \sum_i \frac{|\hat{y}_i - y_i|}{(|\hat{y}_i| + |y_i|)/2}.$$

6. Results (holdout split)

6.1 Per-model metrics

Model	SMAPE (%)	RMSE	MAE	$\mathbf{R^2}$
Ridge (log-price)	53.955	25.990	12.324	0.3477
XGBoost	51.726	25.682	11.506	0.3631
Blend (inv-RMSE)	**50.508**	**25.027**	**11.261**	**0.3951**

Blend weights w (Ridge, XGB): $[0.494, 0.506]$.

6.2 Runtime and artifacts

- **Total runtime:** ∼6525.44 s.
- **Saved predictions:** `dataset/test_out.csv` with columns `sample_id`, `price`.
- **Saved model:** `dataset/final_model.pkl` (models + blend weights). On subsequent runs, training is skipped and the saved model is reused.

7. Conclusion

A robust, memory-safe **sparse** pipeline that fuses engineered structured features with hybrid **Hashing + TF-IDF** text embeddings and **Ridge & XGBoost** ensembling achieves a **holdout SMAPE of 50.508%**, improving over individual models. Caching and model reuse make the solution efficient and fully compliant for the challenge.